Chapter 18
Annotation-Based Controllers

In Chapter 17, "Introduction to Spring MVC" you built two simple Spring MVC applications using some old style controllers. The controllers were classes that implemented the **Controller** interface. Spring 2.5 introduced a new way of creating controllers: by using the **Controller** annotation type.

This chapter discusses annotation-based controllers and the various annotation types that can be beneficial to your applications.

Spring MVC Annotation Types

There are several advantages of using annotation-based controllers. For one, a controller class can handle multiple actions. (By contrast, a controller that implements the **Controller** interface can only handle one action.) This means, related actions can be written in the same controller class, thus reducing the number of classes in your application.

Secondly, with annotation-based controllers request mappings do not need to be stored in a configuration file. Using the **RequestMapping** annotation type, a method can be annotated to make it a request-handling method.

Controller and **RequestMapping** annotation types are two most important annotation types in the Spring MVC API. This chapter focuses on these two and briefly touches on some other less popular annotation types.

The Controller Annotation Type

The **org.springframework.stereotype.Controller** annotation type is used to annotate a Java class to indicate to Spring that instances of the class are controllers. Here is an example of a class annotated with **@Controller**.

```
package com.example.controller;

import org.springframework.stereotype;
...

@Controller
public class CustomerController {

    // request-handling methods here
```

```
}
```

Spring uses a scanning mechanism to find all annotation-based controller classes in an application. To ensure Spring can find your controllers, there are two things you need to do. First, you need to declare the **spring-context** schema in your Spring MVC configuration file, like so:

```
<beans
    ...
    xmlns:context="http://www.springframework.org/schema/context"
    ...
>
```

Second, you need to use a **<component-scan/>** element in your configuration file:

```
context:component-scan base-package="basePackage"/>
```

In your **<component-scan/>** element, specify the base package of your controller classes. For example, if you put all your controller classes under **com.example.controller** and its subpackages, you need to write a **<component-scan/>** element like so.

```
<context:component-scan base-package="com.example.controller"/>
```

Integrating **<component-scan/>**, your configuration file would look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-
       context.xsd">

    <context:component-scan base-package="com.example.controller"/>

    <!-- ... -->
</beans>
```

You would want to make sure all controller classes are part of the base package. At the same time, you don't want to specify a base package that is too wide (say, by specifying **com.example** instead of **com.example.controller**) because this would make Spring MVC scan irrelevant packages.

The RequestMapping Annotation Type

Inside a controller class you write request handling methods that each will handle an action. To tell Spring which method handles which action, you need to map URIs with methods using the **org.springframework.web.bind.annotation.RequestMapping** annotation type.

The **RequestMapping** annotation type does what its name implies: map a

request and a method. You can use **@RequestMapping** to annotate a method or a class.

A method annotated with **@RequestMapping** becomes a request-handling method and will be invoked when the dispatcher servlet receives a request with a matching URI.

Here is a controller class with a **RequestMapping**-annotated method.

```
package com.example.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
...

@Controller
public class CustomerController {

    @RequestMapping(value = "/customer_input")
    public String inputCustomer() {

        // do something here

        return "CustomerForm";
    }
}
```

You specify the URI mapped to the method using the **value** attribute in the **RequestMapping** annotation. In the example above, the URI **customer_input** is mapped with the **inputCustomer** method. This means, the **inputCustomer** method can be invoked using a URL having this pattern.

```
http://domain/context/customer_input
```

Since the **value** attribute is the default attribute of the **RequestMapping** annotation type, you can omit the attribute name if it is the only attribute used in the **RequestMapping** annotation. In other words, these two annotations have the same meaning.

```
@RequestMapping(value = "/customer_input")
```

```
@RequestMapping("/customer_input")
```

However, if more than one attributes appear in **@RequestMapping**, you must write the **value** attribute name.

The value of a request mapping can be an empty string, in which case the method is mapped to the following URL:

```
http://domain/context
```

RequestMapping has other attributes besides **value**. For instance, the **method** attribute takes a set of HTTP methods that will be handled by the corresponding method.

For example, the **processOrder** method below can only be invoked with the HTTP POST or PUT method.

```
...
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
...
    @RequestMapping(value="/order_process",
            method={RequestMethod.POST, RequestMethod.PUT})
    public String processOrder() {

        // do something here

        return "OrderForm";
    }
```

If there is only one HTTP request method assigned to the **method** attribute, the bracket is optional. For instance,

```
@RequestMapping(value="/order_process", method=RequestMethod.POST)
```

If the **method** attribute is not present, the request-handling method will handle any HTTP method.

The **RequestMapping** annotation type can also be used to annotate a controller class like this:

```
import org.springframework.stereotype.Controller;
...

@Controller
@RequestMapping(value="/customer")
public class CustomerController {
```

In this case, request mappings in all methods will be deemed relative to the class-level request mapping. For instance, consider the following **deleteCustomer** method

```
...
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
...
@Controller
@RequestMapping("/customer")
public class CustomerController {

    @RequestMapping(value="/delete",
            method={RequestMethod.POST, RequestMethod.PUT})
    public String deleteCustomer() {

        // do something here

        return ...;
    }
```

Because the controller class is mapped with "/customer" and the **deleteCustomer** method with "/delete", the method can be invoked using a URL with this pattern.

```
http://domain/context/customer/delete
```

Writing Request-Handling Methods

A request-handling method can have a mix of argument types as well as one of a variety of return types. For example, if you need access to the **HttpSession** object in your method, you can add **HttpSession** as an argument and Spring will pass the correct object for you:

```
@RequestMapping("/uri")
public String myMethod(HttpSession session) {
    ...
    session.addAttribute(key, value);
    ...
}
```

Or, if you need the client locale and the **HttpServletRequest**, you can include both as method arguments like this.

```
@RequestMapping("/uri")
public String myOtherMethod(HttpServletRequest request,
        Locale locale) {
    ...
    // access Locale and HttpServletRequest here
    ...
}
```

Here is the list of argument types that can appear as arguments in a request-handling method.

- **javax.servlet.ServletRequest** or **javax.servlet.http.HttpServletRequest**
- **javax.servlet.ServletResponse** or **javax.servlet.http.HttpServletResponse**
- **javax.servlet.http.HttpSession**
- **org.springframework.web.context.request.WebRequest** or **org.springframework.web.context.request.NativeWebRequest**
- **java.util.Locale**
- **java.io.InputStream** or **java.io.Reader**
- **java.io.OutputStream** or **java.io.Writer**
- **java.security.Principal**
- **HttpEntity<?>** parameters
- **java.util.Map / org.springframework.ui.Model / org.springframework.ui.ModelMap**
- **org.springframework.web.servlet.mvc.support.RedirectAttributes**
- **org.springframework.validation.Errors / org.springframework.validation.BindingResult**
- Command or form objects
- **org.springframework.web.bind.support.SessionStatus**
- **org.springframework.web.util.UriComponentsBuilder**
- Types annotated with **@PathVariable**, **@MatrixVariable**, **@RequestParam**, **@RequestHeader**, **@RequestBody**, or **@RequestPart**.

Of special importance is the **org.springframework.ui.Model** type. This is not a Servlet

API type, but rather a Spring MVC type that contains a **Map**. Every time a request-handling method is invoked, Spring MVC creates a **Model** object and populates its **Map** with potentially various objects.

A request-handling method can return one of these objects.

- A **ModelAndView** object
- A **Model** object
- A **Map** containing the attributes of the model
- A **View** object
- A **String** representing the logical view name
- void
- An **HttpEntity** or **ResponseEntity** object to provide access to the Servlet response HTTP headers and contents
- A **Callable**
- A **DeferredResult**
- Any other return type. In this case, the return value will be considered a model attribute to be exposed to the view

You'll learn how to write request-handling methods in the sample applications given later in this chapter.

Using An Annotation-Based Controller

The **app18a** application, a rewrite of the sample applications in Chapter 16 and Chapter 17, presents a controller class with two request-handling methods.

The main difference between **app18a** and the previous applications is the controller class in **app18a** is annotated with **@Controller**. In addition, the Spring configuration file also includes additional elements. Various parts of the application are given in the following subsections.

Directory Structure

Figure 18.1 shows the directory structure of **app18a**. Note that there is only one controller class instead of two. An HTML file, **index.html**, has been added to the application directory to show how static resources can still be accessed when the URL pattern of the Spring MVC servlet is set to /.

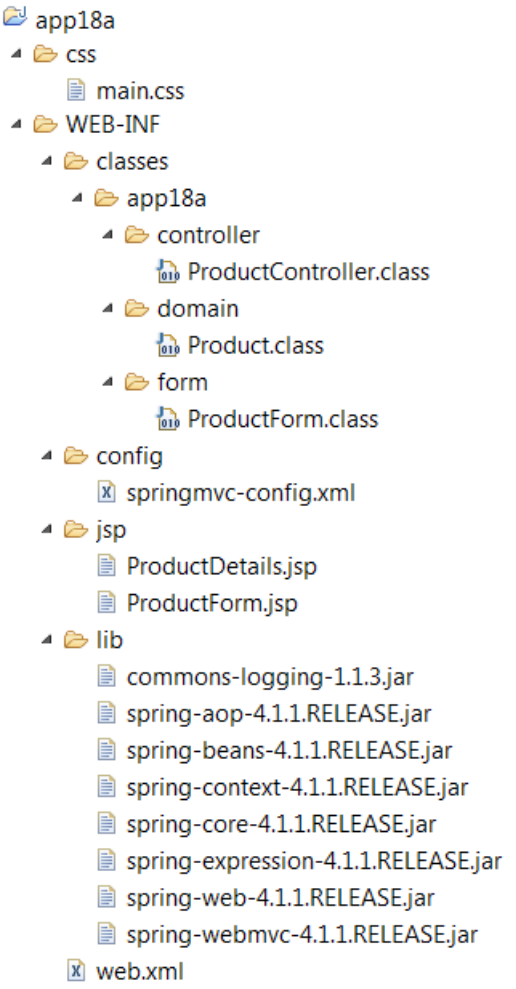

Figure 18.1: The directory structure of app18a

Configuration Files

There are two configuration files in **app18a**. The first, the deployment descriptor (**web.xml** file), registers the Spring MVC dispatcher servlet. The second configuration file, **springmvc-config.xml**, is a Spring MVC configuration file.

Listing 18.1 shows the deployment descriptor and Listing 18.2 the Spring MVC configuration file.

Listing 18.1: The deployment descriptor for app18a (web.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0"
    xmlns="http://java.sun.com/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
➥http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
```

```
    <servlet>
        <servlet-name>springmvc</servlet-name>
        <servlet-class>
            org.springframework.web.servlet.DispatcherServlet
        </servlet-class>
        <init-param>
            <param-name>contextConfigLocation</param-name>
            <param-value>
                /WEB-INF/config/springmvc-config.xml
            </param-value>
        </init-param>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>springmvc</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>
</web-app>
```

Note that in the **<servlet-mapping/>** element in the deployment descriptor, the URL pattern for the Spring MVC dispatcher servlet is set to / as opposed to **.action** as in Chapter 17. This is to show that you do not have to map actions with a certain URL extension. However, setting the URL pattern to / means that all requests, including those for static resources, are directed to the dispatcher servlet. In order for static resources to be handled properly, you need to add some **<resources/>** elements in the Spring MVC configuration file.

Listing 18.2: The springmvc-config.xml file

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-
       context.xsd">
    <context:component-scan base-package="app18a.controller"/>
    <mvc:annotation-driven/>
    <mvc:resources mapping="/css/**" location="/css/"/>
    <mvc:resources mapping="/*.html" location="/"/>

    <bean id="viewResolver"
        class="org.springframework.web.servlet.view.
➥InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/jsp/"/>
```

```
        <property name="suffix" value=".jsp"/>
    </bean>
</beans>
```

The main thing in the Spring MVC configuration file in Listing 18.2 is the presence of a **<component-scan/>** element. It is to tell Spring MVC to scan classes under a certain package, in this case the **app18a.controller** package. On top of that, there are also an **<annotation-driven/>** element and two **<resources/>** elements. The **<annotation-driven/>** element does several things, including registering beans to support request processing with annotated controller methods. The **<resources/>** element tells Spring MVC which static resources need to be served independently from the dispatcher servlet.

In the configuration file in Listing 18.2 there are two **<resources/>** elements. The first makes sure that all files in the **/css** directory will be visible. The second allows displaying of all **.html** files in the application directory.

Note

Without **<annotation-driven/>**, the **<resources/>** elements will prevent any controller from being invoked. You don't need an **<annotation-driven/>** element if you are not using **resources** elements.

The Controller Class

One of the advantages of using the **Controller** annotation type is that a controller class can contain multiple request-handling methods. As can be seen in the **ProductController** class in Listing 18.3, there are two methods in it, **inputProduct** and **saveProduct**.

Listing 18.3: The ProductController class in app18a

```
package app18a.controller;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

import app18a.domain.Product;
import app18a.form.ProductForm;

@Controller
public class ProductController {

    private static final Log logger =
       LogFactory.getLog(ProductController.class);

    @RequestMapping(value="/product_input")
    public String inputProduct() {
        logger.info("inputProduct called");
        return "ProductForm";
    }

    @RequestMapping(value="/product_save")
    public String saveProduct(ProductForm productForm, Model model) {
        logger.info("saveProduct called");
```

```
        // no need to create and instantiate a ProductForm
        // create Product
        Product product = new Product();
        product.setName(productForm.getName());
        product.setDescription(productForm.getDescription());
        try {
            product.setPrice(Float.parseFloat(
                    productForm.getPrice()));
        } catch (NumberFormatException e) {
        }

        // add product

        model.addAttribute("product", product);
        return "ProductDetails";
    }
}
```

Note that the second argument to **saveProduct** in the **ProductController** class is of type **org.springframework.ui.Model**. Spring MVC creates a **Model** instance every time a request-handling method is invoked, whether or not you'll use the instance in your method. The main purpose of having a **Model** is for adding attributes that will be displayed in the view. In this example, you added a **Product** instance by calling **model.addAttribute**:

```
model.addAttribute("product", product);
```

The **Product** instance can then be accessed as if you had added it to the **HttpServletRequest**.

The View

In **app18a** there are two views similar to the ones in previous sample applications, the **ProductForm.jsp** page (given in Listing 18.4) and the **ProductDetails.jsp** page (shown in Listing 18.5).

Listing 18.4: The ProductForm.jsp page

```
<!DOCTYPE html>
<html>
<head>
<title>Add Product Form</title>
<style type="text/css">@import url(css/main.css);</style>
</head>
<body>

<div id="global">
<form action="product_save" method="post">
    <fieldset>
        <legend>Add a product</legend>
        <p>
            <label for="name">Product Name: </label>
            <input type="text" id="name" name="name"
                tabindex="1">
```

```
        </p>
        <p>
            <label for="description">Description: </label>
            <input type="text" id="description"
                name="description" tabindex="2">
        </p>
        <p>
            <label for="price">Price: </label>
            <input type="text" id="price" name="price"
                tabindex="3">
        </p>
        <p id="buttons">
            <input id="reset" type="reset" tabindex="4">
            <input id="submit" type="submit" tabindex="5"
                value="Add Product">
        </p>
    </fieldset>
</form>
</div>
</body>
</html>
```

Listing 18.5: The ProductDetails.jsp page

```
<!DOCTYPE html>
<html>
<head>
<title>Save Product</title>
<style type="text/css">@import url(css/main.css);</style>
</head>
<body>
<div id="global">
    <h4>The product has been saved.</h4>
    <p>
        <h5>Details:</h5>
        Product Name: ${product.name}<br/>
        Description: ${product.description}<br/>
        Price: $${product.price}
    </p>
</div>
</body>
</html>
```

Testing the Application

To test **app18a**, direct your browser to this URL:

```
http://localhost:8080/app18a/product_input
```

You'll see the Product Form in your browser, like the one in Figure 18.2.

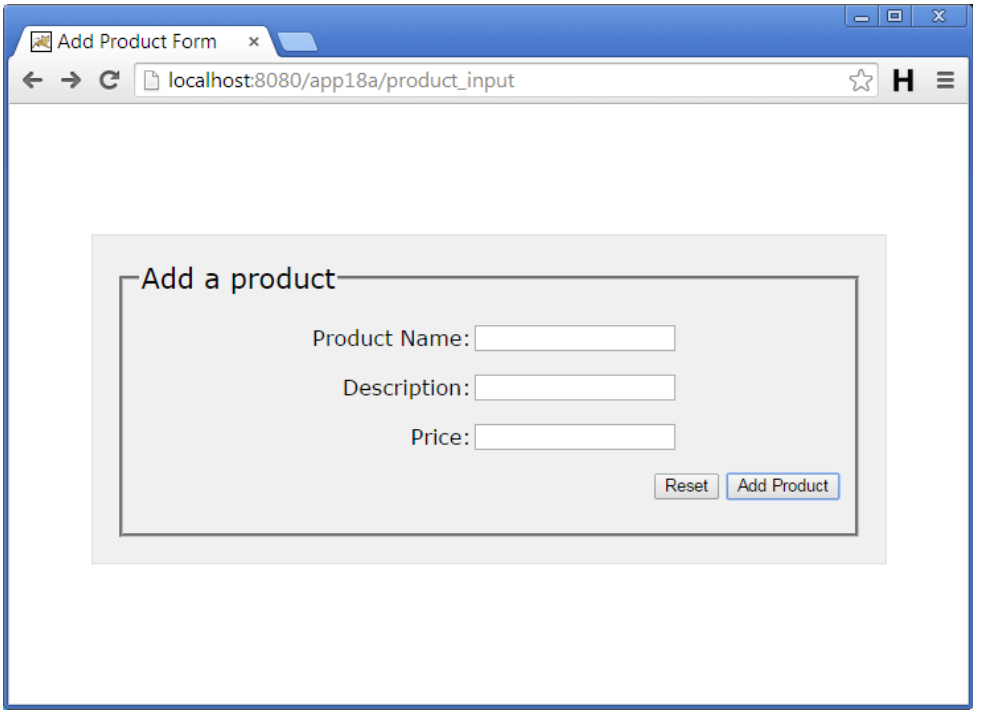

Figure 18.2: The Product form

Pressing the Add Product button will invoke the **saveProduct** method in the controller.

Dependency Injection with @Autowired and @Service

One of the benefits of using the Spring Framework is that you get dependency injection for free. After all, Spring started as a dependency injection container. The easiest way to get a dependency injected to a Spring MVC controller is by annotating a field or a method with **@Autowired**. The **Autowired** annotation type belongs to the **org.springframework.beans.factory.annotation** package.

In order for a dependency to be found, its class must be annotated with **@Service**. A member of the **org.springframework.stereotype** package, the **Service** annotation type indicates that the annotated class is a service. In addition, in your configuration file you need to add a **<component-scan/>** element to scan the base package for your dependencies.

```
<context:component-scan base-package="dependencyPackage"/>
```

As an example of dependency injection in a Spring MVC application, consider another example, the **app18b** application. The **ProductController** class in the **app18b** application (See Listing 18.6) has been modified from the identically named class in **app18a**.

Listing 18.6: The ProductController class in app18b

```
package app18b.controller;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import app18b.domain.Product;
import app18b.form.ProductForm;
import app18b.service.ProductService;

@Controller
public class ProductController {

    private static final Log logger = LogFactory
            .getLog(ProductController.class);

    @Autowired
    private ProductService productService;

    @RequestMapping(value = "/product_input")
    public String inputProduct() {
        logger.info("inputProduct called");
        return "ProductForm";
    }

    @RequestMapping(value = "/product_save", method = RequestMethod.POST)
    public String saveProduct(ProductForm productForm,
            RedirectAttributes redirectAttributes) {
        logger.info("saveProduct called");
        // no need to create and instantiate a ProductForm
        // create Product
        Product product = new Product();
        product.setName(productForm.getName());
        product.setDescription(productForm.getDescription());
        try {
            product.setPrice(Float.parseFloat(
                    productForm.getPrice()));
        } catch (NumberFormatException e) {
        }

        // add product
        Product savedProduct = productService.add(product);

        redirectAttributes.addFlashAttribute("message",
                "The product was successfully added.");
        return "redirect:/product_view/" + savedProduct.getId();
    }
```

```
    @RequestMapping(value = "/product_view/{id}")
    public String viewProduct(@PathVariable Long id, Model model) {
        Product product = productService.get(id);
        model.addAttribute("product", product);
        return "ProductView";
    }
}
```

A couple of things make the **ProductController** class in **app18b** different from its counterpart in **app18a**. The first thing is the addition of the following private field that is annotated with **@Autowired**:

```
@Autowired
private ProductService productService
```

Here, **ProductService** is an interface that provides various methods for handling products. Annotating **productService** with **@Autowired** causes an instance of **ProductService** to be injected to the **ProductController** instance.

Listing 18.7 shows the **ProductService** interface and Listing 18.8 its implementation **ProductServiceImpl**. Note that for the implementation to be scannable, you must annotate its class definition with **@Service**.

Listing 18.7: The ProductService interface

```
package app18b.service
import app18b.domain.Product;
public interface ProductService {
    Product add(Product product);
    Product get(long id);
}
```

Listing 18.8: The ProductServiceImpl class

```
package app18b.service;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;
import org.springframework.stereotype.Service;
import app18b.domain.Product;

@Service
public class ProductServiceImpl implements ProductService {

    private Map<Long, Product> products =
            new HashMap<Long, Product>();
    private AtomicLong generator = new AtomicLong();

    public ProductServiceImpl() {
        Product product = new Product();
        product.setName("JX1 Power Drill");
        product.setDescription(
                "Powerful hand drill, made to perfection");
        product.setPrice(129.99F);
        add(product);
```

```
    }

    @Override
    public Product add(Product product) {
        long newId = generator.incrementAndGet();
        product.setId(newId);
        products.put(newId, product);
        return product;
    }

    @Override
    public Product get(long id) {
        return products.get(id);
    }
}
```

As you can see in Listing 18.9, the Spring MVC configuration file for **app18b** has two **<component-scan/>** elements, one for scanning controller classes and one for scanning service classes.

Listing 18.9: The Spring MVC configuration file for app18b

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-
➥context.xsd">
    <context:component-scan base-package="app18b.controller"/>
    <context:component-scan base-package="app18b.service"/>
    <mvc:annotation-driven/>
    <mvc:resources mapping="/css/**" location="/css/"/>
    <mvc:resources mapping="/*.html" location="/"/>

    <bean id="viewResolver"
       class="org.springframework.web.servlet.view.InternalResourceViewRes
       olver">
        <property name="prefix" value="/WEB-INF/jsp/"/>
        <property name="suffix" value=".jsp"/>
    </bean>
</beans>
```

Redirect and Flash Attributes

As a seasoned servlet/JSP programmer, you must know the difference between and a forward and a redirect. A forward is faster than a redirect because a redirect requires a round-trip to the server and a forward does not. However, there are circumstances where a redirect is preferred. One of such events is when you need to redirect to an external site, like a different web site. You cannot use a forward to target an external site so a redirect is your only choice.

Another scenario where you want to use a redirect instead of a forward is when you want to avoid the same action from being invoked again when the user reloads the page. For example, in **app18a** when you submit the Product form, the **saveProduct** method will be invoked and this method will do what it's supposed to do. In a real-world application, this might include adding the product to the database. However, if you reload the page after you submits the form, **saveProduct** will be called again and the same product would potentially be added the second time. To avoid this, after a form submit, you may prefer to redirect the user to a different page. This page should have no side-effect when called repeatedly. For instance, in **app18a**, you could redirect the user to a **ViewProduct** page after a form submit.

In **app18b** you probably noticed that the **saveProduct** method in the **ProductController** class ends with this line:

```
return "redirect:/product_view/" + savedProduct.getId();
```

Here, you use a redirect instead of a forward to prevent the **saveProduct** method being called twice if the user reloads the page.

The trade-off when using a redirect is you cannot easily pass a value to the target page. With a forward, you can simply add attributes to the **Model** object and the attributes will be accessible to the view. Since a redirect is a round trip to the server, everything in the **Model** is lost when you redirect. Fortunately, Spring version 3.1 and later provide a way of preserving values in a redirect by using flash attributes.

To use flash attributes you must have an **<annotation-driven/>** element in your Spring MVC configuration file. And then, you must also add a new argument of type **org.springframework.web.servlet.mvc.support.RedirectAttributes** in your method. The **saveProduct** method in **ProductController** in **app18b** is reprinted in Listing 18.10.

Listing 18.10: Using flash attributes

```
@RequestMapping(value = "product_save", method = RequestMethod.POST)
public String saveProduct(ProductForm productForm,
        RedirectAttributes redirectAttributes) {
    logger.info("saveProduct called");
    // no need to create and instantiate a ProductForm
    // create Product
    Product product = new Product();
    product.setName(productForm.getName());
    product.setDescription(productForm.getDescription());
    try {
        product.setPrice(Float.parseFloat(productForm.getPrice()));
    } catch (NumberFormatException e) {
```

```
    }

    // add product
    Product savedProduct = productService.add(product);

    redirectAttributes.addFlashAttribute("message",
            "The product was successfully added.");

    return "redirect:/product_view/" + savedProduct.getId();
}
```

Request Parameters and Path Variables

Both request parameters and path variables are used to send values to the server. Both are also part of a URL. The request parameter takes the form of key=value pairs separated by an ampersand. For instance, this URL carries a **productId** request parameter with a value of 3.

```
http://localhost:8080/app18b/product_retrieve?productId=3
```

In servlet programming, you can retrieve a request parameter value by using the **getParameter** method on the **HttpServletRequest**:

```
String productId = httpServletRequest.getParameter("productId");
```

In Spring MVC there is an easier way to retrieve a request parameter value: by using the **org.springframework.web.bind.annotation.RequestParam** annotation type to annotate an argument to which the value of the request parameter will be copied. For example, the following method contains an argument that captures request parameter **productId**.

```
public void sendProduct(@RequestParam int productId)
```

As you can see, the type of argument annotated with **@RequestParam** does not need to be String.

A path variable is similar to a request parameter, except that there is no key part, just a value. For example, the **product_view** action in **app18b** is mapped to a URL with this format.

```
/product_view/productId
```

where *productId* is an integer representing a product identifier. In Spring MVC parlance, *productId* is called a path variable. It is used to send a value to the server.

Consider the **viewProduct** method in Listing 18.11 that demonstrates the use of a path variable.

Listing 18.11: Using path variables

```
@RequestMapping(value = "/product_view/{id}")
public String viewProduct(@PathVariable Long id, Model model) {
    Product product = productService.get(id);
    model.addAttribute("product", product);
    return "ProductView";
```

```
}
```

To use a path variable, first you need to add a variable that is the value attribute of your **RequestMapping** annotation. The variable must be put between curly brackets. For example, the following **RequestMapping** annotation defines a path variable called id:

```
@RequestMapping(value = "/product_view/{id}")
```

Then, in your method signature, add an identically named variable annotated with **@PathVariable**. Take a look at the signature of **viewProduct** in Listing 18.11. When the method is invoked, the **id** value in the request URL will be copied to the path variable and can be used in the method. The type of a path variable does not need to be **String**. Spring MVC will try its best to convert a non-string value. This great feature of Spring MVC's will be discussed in detail in Chapter 19, "Data Binding and the Form Tags."

You can use multiple path variables in your request mapping. For example, the following defines two path variables, **userId** and **orderId**.

```
@RequestMapping(value = "/product_view/{userId}/{orderId}")
```

To test the path variable in the **viewProduct** method, direct your browser to this URL.

```
http://localhost:8080/app18b/product_view/1
```

There is a slight problem when employing path variables: in some cases it may be confusing to the browser. Consider the following URL.

```
http://example.com/context/abc
```

The browser will think (correctly) that **abc** is the action. Any relative reference to a static file, such as a CSS file, will be resolved using http://example.com/context as the base. This is to say, if the page sent by the server contains this **img** element

```
<img src="logo.png"/>
```

The browser will look for **logo.png** in http://example.com/context/logo.png.

However, note that if the same application is deployed as the default context (where the path to the default context is an empty string), the URL for the same target would be this:

```
http://example.com/abc
```

Now, consider the following URL that carries a path variable in an application deployed as the default context:

```
http://example.com/abc/1
```

In ths case, the browser will think **abc** is the context, not the action. If you refer to **** in your page, the browser will look for the image in http://example.com/abc/logo.png and it won't find the image.

Lucky for us there is an easy solution, i.e. by using the JSTL **url** tag. (JSTL was discussed in Chapter 5, "JSTL.") The tag fixes the issue by correctly resolving the URL. For example, all CSS imports in the JSP pages in **app18b** have been changed from

```
<style type="text/css">@import url(css/main.css);</style>
```

to

```
<style type="text/css">
@import url("<c:url value="/css/main.css"/>");
</style>
```

Thanks to the **url** tag, the URL will be translated into this if it is in the default context.

```
<style type="text/css">@import url("/css/main.css");</style>
```

And it will be translated into this if it is not in the default context.

```
<style type="text/css">@import url("/app18b/css/main.css");</style>
```

@ModelAttribute

In the previous section I talked about the **Model** type that Spring MVC creates an instance of every time a request-handling method is invoked. You can add a **Model** as an argument for your method if you intend to use it in your method. The **ModelAttribute** annotation type can be used to decorate a **Model** instance in a method. This annotation type is also part of the **org.springframework.web.bind.annotation** package.

@ModelAttribute can be used to annotate a method argument or a method. A method argument annotated with **@ModelAttribute** will have an instance of it retrieved or created and added to the **Model** object if the method body does not do it explicitly. For example, Spring MVC will create an instance of **Order** every time this **submitOrder** method is invoked.

```
@RequestMapping(method = RequestMethod.POST)
public String submitOrder(@ModelAttribute("newOrder") Order order,
    Model model) {

   ...
}
```

The **Order** instance retrieved or created will be added to the **Model** object with attribute key **newOrder**. If no key name is defined, then the name will be derived from the name of the type to be added to the **Model**. For instance, every time the following method is invoked, an instance of **Order** will be retrieved or created and added to the **Model** using attribute key **order**.

```
public String submitOrder(@ModelAttribute Order order, Model model)
```

The second use of **@ModelAttribute** is to annotate a non-request-handling method. Methods annotated with **@ModelAttribute** will be invoked every time a request-handling method in the same controller class is invoked. This means, if a controller class has two request-handling methods and another method annotated with **@ModelAttribute** method, the annotated method will likely be invoked more often than each of the request-handling methods.

A method annotated with **@ModelAttribute** will be invoked right before a request-handling method. Such a method may return an object or have a void return type. If it returns an object, the object is automatically added to the **Model** that was created for the request-handling method. For example, the return value of this method will be added to

the **Model**.

```
@ModelAttribute
public Product addProduct(@RequestParam String productId) {
    return productService.get(productId);
}
```

If your annotated method returns void, then you must also add a **Model** argument type and add the instance yourself. Here is an example.

```
@ModelAttribute
public void populateModel(@RequestParam String id, Model model) er);
    model.addAttribute(new Account(id));
}
```

Summary

In this chapter you learned how to write Spring MVC applications that use annotation-based controllers. You have also learned various annotation types to annotate your classes, methods, or method arguments.